

Applied Machine Learning

Unit 07 — Lecture 20 Notes

Clustering Foundations

Tofik Ali

Autumn 2026

Read this first

These notes are written so that you can learn the material *without* having been in the lecture. Every concept is developed from scratch, every number you see was produced by running the code shown beside it, and every figure was generated from real computation rather than drawn by hand.

Everything you have built so far had an answer key. Regression had a numeric target; classification had a class column. Every choice — which model, how deep, how many neighbours — was settled by scoring predictions against a column that was already sitting in the file. Cross-validation, the confusion matrix, ROC and AUC: none of them exist without that column.

This unit removes it. The data arrives as n rows and d features and nothing else, and two things that you have never had to think about must now be rebuilt from scratch: **what counts as a good answer**, and **how you would know**.

Three results carry the lecture and you should be able to reproduce all three with a pen. The first is Section 3: one dataset, **two defensible clusterings that disagree** — the silhouette prefers $k = 2$ on iris at 0.6810, the species prefer $k = 3$, and the two clusterings agree with each other at ARI only 0.5516. The second is Section 5.3: the identity $\mathbf{T} = \mathbf{W} + \mathbf{B}$, worked on six points where $91 = 16 + 75$ for one clustering and $91 = 79 + 12$ for another. The third is Section 6.2: the silhouette of a single point computed by hand, $s(P_1) = \mathbf{0.691137}$, agreeing with `scikit-learn` to six decimal places. Those three are the examinable content of this lecture.

Contents

1	How to use these notes	2
1.1	What you need	3
1.2	Learning outcomes	4
2	What changes when there is no target column	4
2.1	The setup, and the vocabulary	4
2.2	Notation, and the kinds of clustering	5
2.3	Why nothing searches exhaustively	5
3	There is no single right answer	6
3.1	Two clusterings of iris, both defensible	6
3.2	The same lesson again: scaling is the definition of “similar”	7

3.3	What “exploratory” means	8
4	The taxonomy of clustering algorithms	9
4.1	Five families	9
4.2	Measured: four shapes, five algorithms	9
4.3	A grid clusterer, built from scratch	11
5	The statistics of cluster analysis	12
5.1	The cluster centroid	12
5.2	Within, between, and total	13
5.3	The decomposition, and why it matters	13
5.4	Six points, by hand	14
5.5	Verified on real data, at every k	15
5.6	Why W cannot choose k	16
5.7	Calinski–Harabasz: the decomposition with the penalty put back	16
6	Internal and external validation	17
6.1	Two kinds of validation, and only one is usually available	17
6.2	The silhouette, by hand	17
6.3	Silhouette on real data, and the profile	19
6.4	Davies–Bouldin	19
6.5	All three across k , with no labels used	20
6.6	External validation: ARI and NMI	20
6.7	The honest disagreement	21
7	Clustering as a preprocessing tool	22
7.1	Cluster membership as an engineered feature	22
7.2	And the null result, reported as it happened	23
7.3	Cluster-then-label: spending a labelling budget	23
7.4	The leakage rule	25
8	General applications of clustering	25
9	Summary	27
10	Practice questions	28
11	Solutions	29
12	References and further reading	32

1 How to use these notes

Read in order. Sections 2 to 6 are one continuous argument — what changes without a target, why the answer is not unique, what the algorithm families commit to, what quantities describe a clustering, and how to judge one — and Section 7 onwards does not make sense without them.

Everything here assumes the supervised units: train/test splits, the leakage rule from the preprocessing lecture, and the habit of measuring a claim before making it. Two ideas come back in new clothes. Feature scaling, which in a supervised model costs you accuracy the test set reports, here silently *redefines the question* and nothing complains (Section 3.2). And leakage, which you met as “fit the scaler on the training rows only”, reappears as “fit the clusterer on the training rows only” (Section 7.4).

There are four kinds of box. **Key idea** — the one sentence to remember a month from now. **Worked example** — a calculation done by hand, with small numbers, before any library is used. **Caution** — the specific mistake that section exists to prevent. **Try it yourself** — something to run, with the expected output given so you can check.

Code appears in a framed block, and directly beneath it a shaded block showing what that code *actually printed*. If you run it and get something different, trust your machine and tell me.

1.1 What you need

```
pip install numpy scipy scikit-learn matplotlib
```

Every number in these notes was produced with:

```
import numpy, scipy, sklearn, matplotlib
print("numpy", numpy.__version__, "| scipy", scipy.__version__,
      "| sklearn", sklearn.__version__,
      "| matplotlib", matplotlib.__version__)
```

```
numpy 2.2.6 | scipy 1.15.3 | sklearn 1.7.2 | matplotlib 3.10.9
```

The datasets recur, and **none of them downloads anything**:

```
from sklearn.datasets import (load_iris, load_wine, load_digits,
                              load_breast_cancer, make_blobs, make_moons)
iris = load_iris(); X, y = iris.data, iris.target
```

```
load_iris      150 rows x  4 features, 3 known groups
load_wine      178 rows x 13 features, 3 known groups
load_digits    1797 rows x 64 features, 10 known groups
load_breast_cancer 569 rows x 30 features, 2 known groups
make_moons     500 rows x  2 features, 2 known groups
missing cells anywhere: 0
labels are used ONLY to score a clustering, never to fit one
```

That last line is the rule for the whole unit. Several of these datasets ship with a label column. **No clustering in these notes is ever fitted using it.** The labels appear only after the fact, as a way of checking whether an unsupervised score was telling the truth — which is a luxury you will not have on real data, and which is exactly why we exploit it here.

Do not reach for fetch_*

Every `fetch_*` loader downloads over HTTPS, and on the campus network those downloads fail with a certificate error you cannot fix from your machine. Nothing in this course depends on one: use the bundled loaders or a fixed-seed synthetic set.

One seed for the lecture, and three that fix the data

Clustering has no target column to anchor it, so *every* number below depends on where the algorithm started. Two seeds are therefore stated explicitly and never hidden in a helper.

The one arbitrary seed is 0. Every `KMeans` and `GaussianMixture` start, every `train_test_split` and every noise draw in these notes is written `random_state=0` or `np.random.default_rng(0)`. It is arbitrary; what matters is that there is exactly one of it and that you can see it.

The dataset seeds are different in kind. `random_state=170` for the blobs, `random_state=7` for the two moons and `random_state=3` for the noisier moons of Section 7 do not add randomness to an experiment — they *define which points exist*. Change one and you are clustering different data, so every figure and every number moves with it.

So every number printed in these notes reproduces *exactly* if you run the code as written. If yours differs, check the seed before you doubt the notes — and note that `KMeans` needs *both* `n_init` and `random_state` pinned, because the default `n_init` has changed between scikit-learn versions.

1.2 Learning outcomes

These notes serve **CO1** — *understand the foundations of machine learning, the types of learning problem, and the vocabulary used to describe them*. By the end you should be able to:

1. State what clustering is, contrast it with classification on at least four axes, and explain why “what is the right answer” has no meaning here. **CO1**
2. Give a dataset on which two different clusterings are both defensible, and quantify their disagreement. **CO1**
3. Name the five families of clustering algorithm and state what each assumes about the shape of a cluster. **CO1**
4. Given a small dataset and a proposed clustering, compute the cluster centroids, W , B and T by hand, and verify $T = W + B$. **CO1**
5. Explain why T does not depend on the clustering, and deduce that minimising W and maximising B are the same problem. **CO1**
6. Explain why W cannot be used to choose k , and compute Calinski–Harabasz from W , B , k and n . **CO1**
7. Compute $a(i)$, $b(i)$ and $s(i)$ for a point by hand, and interpret a negative silhouette. **CO1**
8. Distinguish internal from external validation, state which is available in practice, and explain why the Rand index must be corrected for chance. **CO1**
9. Describe two ways clustering is used as a preprocessing step, and state the leakage rule that governs both. **CO1**
10. Name four concrete application areas and say, for each, what a “cluster” corresponds to. **CO1**

2 What changes when there is no target column

2.1 The setup, and the vocabulary

A supervised problem hands you pairs (x_i, y_i) and asks you to predict y from x . An unsupervised problem hands you only the x_i and asks you to say something about their structure. **Clustering** is the particular unsupervised task of dividing the objects into groups such that objects in the same group are similar to one another and objects in different groups are not.

That definition is the one the examination will ask for, and every word of it is a decision you have to make:

- *similar* — similar in what sense? Euclidean distance on raw columns? On standardised columns? Cosine similarity? Something for categorical attributes? The next lecture is entirely about this word.

- *groups* — how many? Must every object be in one? Can an object be in two?
- *such that* — how much better is one grouping than another, and according to what?

	Supervised	Unsupervised
Input	$(x_i, y_i), i = 1 \dots n$	x_i only
Goal	predict y for a new x	find structure among the x_i
Objective	given (squared error, log loss)	chosen by you
A correct answer	exists, and is in the file	does not exist
Validation	held-out score	internal indices, then judgement
Failure looks like	high test error	a partition nobody can use

Key idea

In supervised learning the objective is handed to you by the problem. In clustering the objective **is** the modelling decision. Two people who choose differently will get different clusters from the same data, and neither of them is making a mistake.

2.2 Notation, and the kinds of clustering

Write $D = \{x_1, \dots, x_n\}$ with $x_i \in \mathbb{R}^d$. A **clustering** into k groups is a map $C : D \rightarrow \{1, \dots, k\}$; write $C_j = \{x_i : C(x_i) = j\}$ for the j th cluster and $n_j = |C_j|$ for its size. The default in this course is **hard, exhaustive and exclusive**:

$$\bigcup_{j=1}^k C_j = D, \quad C_a \cap C_b = \emptyset \quad (a \neq b), \quad n_j \geq 1.$$

Every object belongs to exactly one cluster. This is what k-means and agglomerative clustering produce, and it is the setting for every calculation below. Three variants matter enough to name:

- **Soft** (or fuzzy, or probabilistic) — each object receives k membership weights summing to one. A Gaussian mixture model produces this, and you recover a hard clustering by taking the largest weight.
- **Partial** — some objects belong to no cluster at all. DBSCAN produces this, and calls the leftovers *noise*. This is a feature, not a defect: see Section 8.
- **Hierarchical** — not one clustering but a nested family, one per level, from n singletons up to a single group. A later lecture in this unit builds it.

2.3 Why nothing searches exhaustively

The number of ways to partition n objects into exactly k non-empty groups is the Stirling number of the second kind $S(n, k)$. For $n = 25$ and $k = 3$ that is 141 197 991 025; for $n = 150$ and $k = 3$ it is a number with 71 digits. Finding the partition of minimum within-cluster sum of squares is NP-hard in general.

So every algorithm in this unit is a **heuristic**. k-means finds a local optimum and depends on its starting centres. Agglomerative clustering makes a greedy merge and never reconsiders it. DBSCAN does not optimise anything at all. When you read that k-means “minimises the within-cluster sum of squares”, read it as “descends towards a local minimum of”, and remember why `n_init` exists.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine*

3 There is no single right answer

3.1 Two clusterings of iris, both defensible

`load_iris` has 150 flowers and four measurements. Throw the species column away, run k-means for $k = 2$ and for $k = 3$, and score both.

```
from sklearn.cluster import KMeans
from sklearn.metrics import (silhouette_score, calinski_harabasz_score,
                             davies_bouldin_score, adjusted_rand_score)
two = KMeans(n_clusters=2, n_init=10, random_state=0).fit_predict(X)
three = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(X)
print(silhouette_score(X, two), silhouette_score(X, three))
print(adjusted_rand_score(y, two), adjusted_rand_score(y, three))
print(adjusted_rand_score(two, three))
```

	k = 2	k = 3
silhouette	0.6810	0.5528
Calinski-Har.	513.92	561.63
Davies-Bould.	0.4043	0.6620
ARI vs labels	0.5399	0.7302

agreement between the two clusterings: ARI 0.5516

Read the four rows. The **silhouette**, which rewards clusters that are tight and far apart, prefers $k = 2$ by a wide margin (0.6810 against 0.5528). **Davies–Bouldin**, where lower is better, also prefers $k = 2$. **Calinski–Harabasz** prefers $k = 3$. And the **adjusted Rand index against the true species** prefers $k = 3$ decisively. Section 6 defines all four; for now only the disagreement matters.

The last line is the one to sit with. The two clusterings agree at ARI **0.5516** — they are substantially different partitions of the same 150 flowers, and there is a respectable argument for each.

```
for name, lab in (("k = 2", two), ("k = 3", three)):
    tab = np.zeros((3, len(np.unique(lab))), dtype=int)
    for t, c in zip(y, lab): tab[t, c] += 1
    print(name, tab)
```

k = 2			
setosa	50	0	
versicolor	3	47	
virginica	0	50	
k = 3			
setosa	0	50	0
versicolor	48	0	2
virginica	14	0	36

At $k = 2$, cluster 0 is almost exactly *setosa* and cluster 1 is *versicolor* and *virginica* stuck together. At $k = 3$ they come apart, but imperfectly: 14 *virginica* land in the *versicolor* cluster.

Key idea

The silhouette is not wrong about iris. It is answering its own question — *are these clusters well separated?* — correctly, and the honest answer is that *versicolor* and *virginica* are not well separated in these four measurements. It is simply not answering the botanist’s

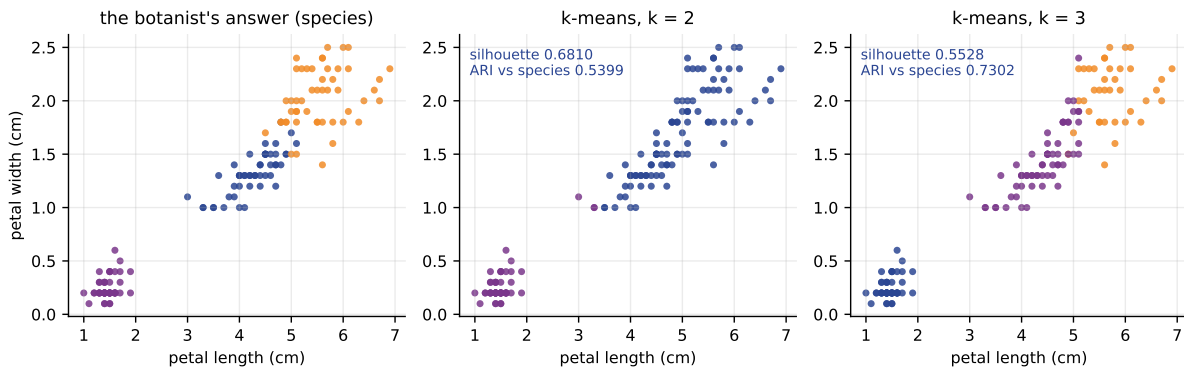


Figure 1: One dataset, three groupings. Left: the species, which a botanist supplies from outside the data. Middle and right: what k-means returns for $k = 2$ and $k = 3$. *Setosa* is genuinely far from the other two, and the other two genuinely touch — so a score that rewards separation merges them and a score that knows the species does not.

question. When an internal index and a set of labels disagree, the useful move is to ask what each of them is measuring, not to declare one of them broken.

3.2 The same lesson again: scaling is the definition of “similar”

The iris example is subtle because both answers are close. Here is a blunter one. `load_wine` has 178 bottles, 13 chemical measurements, and three known cultivars. Run k-means with $k = 3$ twice: once on the raw table, once after standardising every column.

```
from sklearn.preprocessing import StandardScaler
raw = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(Xw)
std = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(
    StandardScaler().fit_transform(Xw))
```

```
column ranges in the raw table
  alcohol      min    11.03  max    14.83  sd      0.81
  magnesium    min    70.00  max   162.00  sd     14.24
  color_intensity min    1.28  max    13.00  sd      2.31
  proline      min   278.00  max  1680.00  sd    314.02
unscaled: silhouette in its own space 0.5711, ARI vs cultivar 0.3711
scaled : silhouette in its own space 0.2849, ARI vs cultivar 0.8975
agreement between the two clusterings: ARI 0.3543
```

Standardising takes the agreement with the true cultivar from **0.3711** to **0.8975**, and the two clusterings agree with each other at only 0.3543 — barely better than half. One preprocessing decision, made in one line, produced two substantially different pictures of the same 178 bottles.

The reason is arithmetic:

```
share of the raw total variance held by each column
  proline      98609.60 (99.77%)
  magnesium     202.84 ( 0.21%)
  alkalinity_of_ash 11.09 ( 0.01%)
  color_intensity   5.34 ( 0.01%)
top column alone accounts for 99.77% of the distance budget
```

`proline` runs into the thousands while `alcohol` runs from 11 to 15. Squared Euclidean distance on the raw table is **99.77%** a comparison of proline; the other twelve columns are decoration.

The silhouettes above are not comparable

0.5711 and 0.2849 were computed in *different spaces*. An internal index measures how good a partition is *given a distance*; change the distance and you have changed the ruler as well as the thing being measured. Never compare an internal index across two different feature scalings, two different distance metrics, or two different feature subsets — and be suspicious of anybody who does.

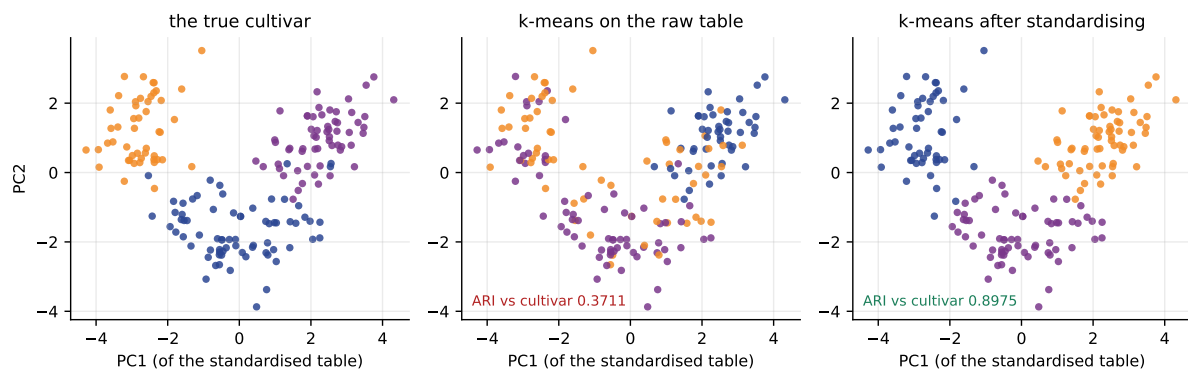


Figure 2: The same 178 bottles, projected onto the first two principal components of the standardised table so that all three panels are drawn in one space. Left, the cultivars. Middle, k-means on the raw table, which is substantially a one-dimensional split on proline. Right, k-means after standardising.

3.3 What “exploratory” means

Put the two demonstrations together. Clustering does not *discover* a grouping that was already there; it *proposes* one, and the proposal is a joint consequence of the data, the distance, the algorithm and the settings. Vladimir Estivill-Castro put it in one line in 2002: *clustering is in the eye of the beholder*.

That is not a licence to be sloppy — it is the opposite. Because there is no answer key, the burden of stating your assumptions falls entirely on you. A defensible clustering result is one that comes with:

1. the distance used, and the scaling that produced it;
2. the algorithm and the settings, including how k was chosen;
3. at least two internal indices, *including the ones that disagreed*;
4. a look at the actual clusters — their sizes, their centroids, a few members of each;
5. a statement of what the groups will be used for.

Try it yourself

Run the wine comparison yourself and add a third variant: scale with `MinMaxScaler` instead of `StandardScaler`. You should find a third clustering, different again from both. Then ask the question these notes keep asking: which one is right? There is no answer, and noticing that there is no answer is the point of this section.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Kriegel, Sander and Xu, “A density-based algorithm for discovering clusters (DBSCAN),” KDD 1996.

4 The taxonomy of clustering algorithms

4.1 Five families

The literature contains hundreds of clustering algorithms. They fall into five families, and the families differ in what they *assume a cluster looks like*. That assumption is the thing to remember; the implementation details come in the later lectures.

Family	Mechanism	Assumed shape
Partitioning	choose k representatives, assign each point to the nearest, move the representatives, repeat until stable. <i>k-means, k-medoids, CLARA</i> .	convex, roughly spherical, roughly equal size; one centre each. The boundary between two clusters is always a hyperplane.
Hierarchical	repeatedly merge the two closest clusters (agglomerative) or repeatedly split (divisive), producing a dendrogram. <i>HAC with single, complete, average or Ward linkage</i> .	whatever the linkage assumes. Ward behaves like k-means; single linkage assumes clusters are <i>connected chains</i> and will happily produce a long thin one.
Density-based	a cluster is a maximal set of points connected through regions whose local density exceeds a threshold. <i>DBSCAN, OPTICS, DENCLUE</i> .	dense regions separated by sparse ones. Shape is unconstrained. Density is assumed roughly uniform within a run. Points in sparse regions are left out as noise.
Grid-based	quantise the space into cells, summarise each cell, then cluster the cells. <i>STING, CLIQUE, WaveCluster</i> .	as density-based, plus the assumption that a <i>fixed resolution</i> is adequate. Fast: cost is driven by the number of cells, not the number of points.
Model-based	assume the data was generated by a mixture of distributions and fit the mixture by maximum likelihood, usually with EM. <i>Gaussian mixture models, latent class models</i> .	whatever family you assumed. A full-covariance Gaussian mixture assumes ellipsoidal clusters of arbitrary orientation and size, which is strictly more general than k-means.

4.2 Measured: four shapes, five algorithms

A taxonomy is only worth learning if the distinctions do something. Here are four two-dimensional problems, each with a known generating group structure, and five algorithms run on each. The score is the adjusted Rand index against the generating groups — 1.0000 is perfect, 0.0000 is chance.

```
Xb, yb = make_blobs(n_samples=500, centers=3, cluster_std=1.0,
                    random_state=170) # spherical
Xa, ya = make_blobs(n_samples=500, random_state=170)
Xa = Xa @ np.array([[0.60, -0.60], [-0.40, 0.80]]) # elongated
Xv, yv = make_blobs(n_samples=500, cluster_std=[1.0, 2.5, 0.5],
                    random_state=170) # unequal spread
Xm, ym = make_moons(n_samples=500, noise=0.06, random_state=7)
```

dataset (ARI)	KMeans	Ward	Single	GaussMix	DBSCAN
spherical blobs	1.0000	1.0000	0.5682	1.0000	0.9910
elongated blobs	0.5554	0.5281	0.5682	1.0000	0.9970
unequal spread	0.7872	0.8692	0.0002	0.9468	0.8054
two moons	0.2445	0.3447	1.0000	0.5003	1.0000

no column wins every row

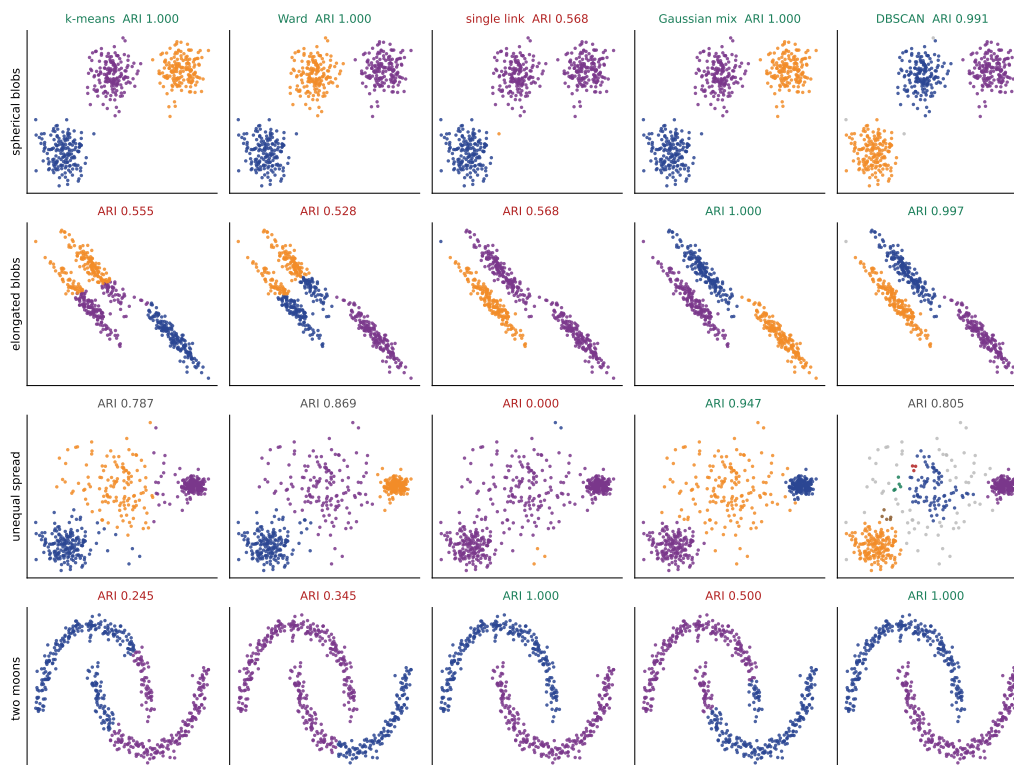


Figure 3: The taxonomy with teeth. Rows are the four problems, columns the five algorithms; each panel is titled with its adjusted Rand index, green above 0.9 and red below 0.6. Every column contains at least one red panel.

Four things to take from that table.

k-means fails on the moons, and fails confidently. It scores 0.2445. The reason is structural rather than a matter of tuning: k-means assigns each point to the nearest centre, so the boundary between two clusters is the perpendicular bisector of their centres — a straight line. Two interleaved crescents cannot be separated by a straight line. What makes this dangerous is that k-means does not complain. It returns two clean-looking clusters and no warning.

DBSCAN succeeds on the moons, exactly. ARI 1.0000. It never asks where the centre of a cluster is; it grows a cluster through any connected dense region, and a crescent is a connected dense region.

The Gaussian mixture succeeds on the elongated blobs where k-means fails. 1.0000 against 0.5554. Rotating and stretching the blobs breaks k-means’ spherical assumption but not the mixture’s ellipsoidal one. This is the clearest illustration of what “model-based” buys: a strictly larger family of admissible cluster shapes.

Single linkage is a knife with no handle. Perfect on the moons (1.0000), catastrophic when the clusters have unequal spread (0.0002, which is chance). It merges whichever two clusters have the single closest pair of points, so one stray point in the gap *chains* two clusters into one. On the wide cluster of the third dataset that happens immediately.

And DBSCAN’s own weakness is in the same row: 0.8054 on unequal spread, its worst result, with 68 of 500 points discarded as noise.

spherical blobs	eps=1.00 -> 3 clusters,	3 points called noise
elongated blobs	eps=0.50 -> 3 clusters,	1 points called noise
unequal spread	eps=0.75 -> 6 clusters,	68 points called noise

```
two moons          eps=0.20 -> 2 clusters,    0 points called noise
```

A single density threshold cannot fit clusters of genuinely different density: set it loose enough for the sparse cluster and the dense ones merge; set it tight enough for the dense ones and the sparse one dissolves into noise. Here it found six clusters where there are three.

Key idea

There is no default clustering algorithm. Choosing one is choosing a shape assumption, and the assumption is a claim about your data that you should be able to defend. If you cannot say what shape you expect your clusters to be, plot the data before you cluster it.

4.3 A grid clusterer, built from scratch

Grid methods sound exotic and are not. Here is a complete one, in three steps: lay a $G \times G$ grid over the data, keep the cells containing at least τ points, and treat each connected group of surviving cells as a cluster.

```
def grid_cluster(X, G=10, tau=3):
    lo, hi = X.min(axis=0) - 1e-9, X.max(axis=0) + 1e-9
    idx = np.clip(((X - lo) / (hi - lo) * G).astype(int), 0, G - 1)
    counts = np.zeros((G, G), dtype=int)
    for a, b in idx: counts[a, b] += 1
    dense = counts >= tau
    comp, ncomp = ndimage.label(dense, structure=np.ones((3, 3)))
    labels = np.array([comp[a, b] - 1 if dense[a, b] else -1 for a, b in idx])
    return labels, counts, dense, ncomp
```

```
cell counts, x across, y up (a dot = fewer than tau = 3)
. . . 13 21 10 . . . .
. 17 8 6 20 12 . . . .
4 11 . . . 17 3 . . .
17 3 . 7 . 5 11 . . 6
14 . . 12 . . 11 . . 15
12 . . 16 . . 16 . . 11
9 . . 10 6 . 6 . . 19
. . . 4 16 . . . 16 4
. . . . 14 19 6 17 14 .
. . . . . 5 19 9 . .
connected dense regions: 2
```

You can see the two crescents in the printed table with your eyes. Forty-three of the hundred cells survive the threshold and they fall into exactly two connected regions, giving ARI **0.9643** with nine points discarded as noise. The whole algorithm is a two-dimensional histogram plus a flood fill, and it never computes a single pairwise distance.

That is the advantage of the family: the cost depends on the number of *cells*, not the number of points, so a grid method scales to data that a distance-matrix method cannot touch. The disadvantage is on the next line.

G=10 tau=3	dense cells	43/100	clusters	2	noise	9	ARI	0.9643
G=12 tau=2	dense cells	56/144	clusters	1	noise	9	ARI	0.0000
G=14 tau=2	dense cells	73/196	clusters	2	noise	13	ARI	0.9486
G=14 tau=3	dense cells	62/196	clusters	2	noise	35	ARI	0.8646

A grid clusterer is brittle at its resolution

Loosen the threshold by one point, from $\tau = 3$ to $\tau = 2$ at $G = 12$, and a single cell between the two arms becomes dense. The two regions connect, the answer becomes “one cluster”, and the ARI collapses from 0.9643 to **0.0000**. Nothing warned; the algorithm ran successfully and returned a wrong answer. Whenever you use a method with a resolution parameter, run it at several resolutions and look at the whole range before believing any single one.

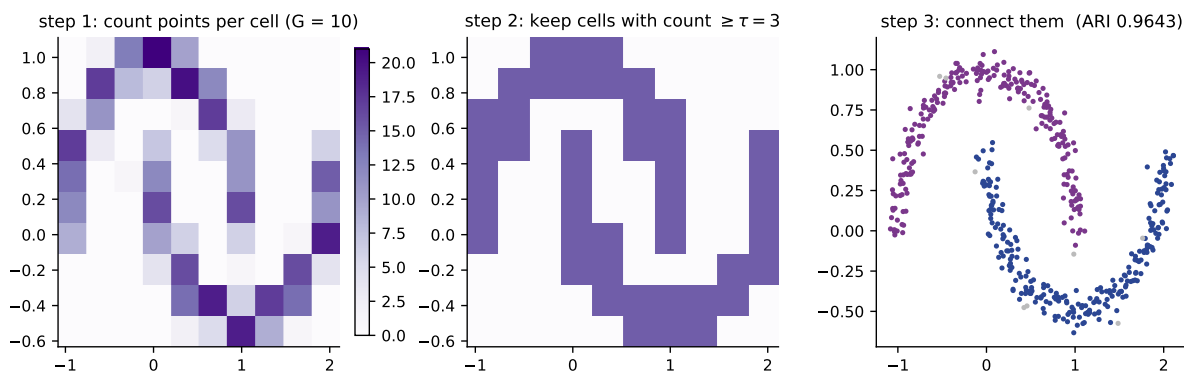


Figure 4: A grid clusterer in three steps: count, threshold, connect.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Kriegel, Sander and Xu, “A density-based algorithm for discovering clusters (DBSCAN),” KDD 1996.

5 The statistics of cluster analysis

Whatever algorithm produced a clustering, the same handful of quantities describe it. They are the vocabulary of the rest of the unit and the source of most examination arithmetic.

5.1 The cluster centroid

The **centroid** of cluster C_j is the componentwise mean of its members,

$$c_j = \frac{1}{n_j} \sum_{x \in C_j} x. \quad (1)$$

It is a point in $\mathbb{R}^d$ and it need not be one of the data points — the centroid of two flowers is generally not a flower. (When you insist that the representative *be* a data point you get the **medoid** instead, and the algorithm becomes k-medoids, which a later lecture in this unit covers.)

The centroid has a property that everything below depends on: it is the point that minimises the sum of squared distances to the members of the cluster. For any $m \in \mathbb{R}^d$,

$$\sum_{x \in C_j} \|x - m\|^2 = \sum_{x \in C_j} \|x - c_j\|^2 + n_j \|c_j - m\|^2. \quad (2)$$

The second term is non-negative and vanishes only at $m = c_j$. We prove this in Section 5.3; it is a two-line expansion and it does all the work.

```
km3 = KMeans(n_clusters=3, n_init=10, random_state=0).fit(X)
print(km3.cluster_centers_, X.mean(axis=0))
```

cluster centroids on the four iris measurements					
	sepal len	sepal wid	petal len	petal wid	n
c0	5.901613	2.748387	4.393548	1.433871	62
c1	5.006000	3.428000	1.462000	0.246000	50
c2	6.850000	3.073684	5.742105	2.071053	38
grand	5.843333	3.057333	3.758000	1.199333	

A centroid is also the most useful *description* of a cluster you have. Cluster **c1** has petals of length 1.46 cm; the other two have petals of 4.39 and 5.74 cm. That single column says immediately what the clustering found. When you report a clustering, report the centroids.

5.2 Within, between, and total

Three sums of squares. Let $\bar{x}$ be the grand mean of all n points.

$$T = \sum_{i=1}^n \|x_i - \bar{x}\|^2 \quad \text{total scatter: a property of the data alone} \quad (3)$$

$$W = \sum_{j=1}^k \sum_{x \in C_j} \|x - c_j\|^2 \quad \text{within-cluster sum of squares (WCSS): tightness} \quad (4)$$

$$B = \sum_{j=1}^k n_j \|c_j - \bar{x}\|^2 \quad \text{between-cluster sum of squares (BCSS): separation} \quad (5)$$

Note carefully what each does and does not depend on. T contains no reference to the clustering at all — change C and T does not move. W is small when the clusters are tight. B is large when the centroids are spread out, and it weights each centroid by the size of its cluster, so moving a large cluster away from the grand mean contributes more than moving a small one.

5.3 The decomposition, and why it matters

The identity everything rests on

$$T = W + B.$$

The total scatter of a dataset splits exactly into a within-cluster part and a between-cluster part, for *any* clustering. Since T is fixed by the data, **minimising W and maximising B are the same problem**. That is why k-means only ever mentions W , and why every index in Section 6 is built out of these two numbers.

The proof

Take a single cluster C_j and put $m = \bar{x}$ in equation (2). Expanding the left side directly:

$$\sum_{x \in C_j} \|x - \bar{x}\|^2 = \sum_{x \in C_j} \|(x - c_j) + (c_j - \bar{x})\|^2 = \sum_{x \in C_j} \|x - c_j\|^2 + 2(c_j - \bar{x})^\top \sum_{x \in C_j} (x - c_j) + n_j \|c_j - \bar{x}\|^2.$$

The middle term vanishes, because $\sum_{x \in C_j} (x - c_j) = 0$ by the definition of the centroid in equation (1) — that is the *only* property of the centroid used anywhere in this argument. So

$$\sum_{x \in C_j} \|x - \bar{x}\|^2 = \sum_{x \in C_j} \|x - c_j\|^2 + n_j \|c_j - \bar{x}\|^2.$$

Now sum over $j = 1, \dots, k$. The clusters partition the data, so the left side sums to T ; the right side sums to $W + B$. $\square$

The proof also tells you when the identity fails: it needs c_j to be exactly the mean of C_j . Compute W around medoids, or around centroids from a previous iteration, and $T = W + B$ stops holding. If your numbers do not add up, that is the first thing to check.

5.4 Six points, by hand

The whole decomposition on six points

Take six points in the plane:

$$P_1(2, 2) \quad P_2(4, 2) \quad P_3(3, 5) \quad P_4(9, 3) \quad P_5(11, 3) \quad P_6(10, 6).$$

The grand mean is $\bar{x} = (\frac{2+4+3+9+11+10}{6}, \frac{2+2+5+3+3+6}{6}) = (6.5, 3.5)$.

Clustering A — left three against right three.

$$c_1 = \left(\frac{2+4+3}{3}, \frac{2+2+5}{3}\right) = (3, 3), \quad c_2 = \left(\frac{9+11+10}{3}, \frac{3+3+6}{3}\right) = (10, 4).$$

Within, cluster by cluster. Offsets from $(3, 3)$ are $(-1, -1)$, $(1, -1)$, $(0, 2)$, contributing $2 + 2 + 4 = 8$. Offsets from $(10, 4)$ are the same three vectors, contributing another 8. So

$$W_A = 8 + 8 = \mathbf{16}.$$

Between. Both centroids sit at squared distance $3.5^2 + 0.5^2 = 12.25 + 0.25 = 12.5$ from the grand mean, and each has three members:

$$B_A = 3(12.5) + 3(12.5) = \mathbf{75}.$$

Total, computed independently from the grand mean:

$$T = 22.5 + 8.5 + 14.5 + 6.5 + 20.5 + 18.5 = \mathbf{91},$$

taking the six terms in order P_1 to P_6 . And $16 + 75 = 91$. $\checkmark$

Clustering B — low y against high y . Put $\{P_1, P_2, P_4, P_5\}$ in one cluster and $\{P_3, P_6\}$ in the other:

$$c_1 = (6.5, 2.5), \quad c_2 = (6.5, 5.5).$$

Within: $20.5 + 6.5 + 6.5 + 20.5 = 54$ for the first, $12.5 + 12.5 = 25$ for the second, so $W_B = \mathbf{79}$. Between: $4 \times 1 + 2 \times 4 = \mathbf{12}$. And $79 + 12 = 91$ again.

That pair of calculations is the whole section. **The same T ; two completely different splits of it.** Clustering A is tight and well separated ($16 \mid 75$); clustering B is loose and overlapping ($79 \mid 12$). The decomposition is the instrument that says so, and it is the reason we can rank two clusterings at all without an answer key.

```
def scatter_decomposition(X, labels):
    g = X.mean(axis=0)
    total = ((X - g) ** 2).sum()
    within = between = 0.0
    for c in np.unique(labels):
        M = X[labels == c]; ck = M.mean(axis=0)
        within += ((M - ck) ** 2).sum()
        between += len(M) * ((ck - g) ** 2).sum()
```

```
return total, within, between
```

clustering	T	W	B	W+B
A: left/right	91.00	16.00	75.00	91.00
B: low/high y	91.00	79.00	12.00	91.00

T does not depend on the clustering; W and B trade off inside it

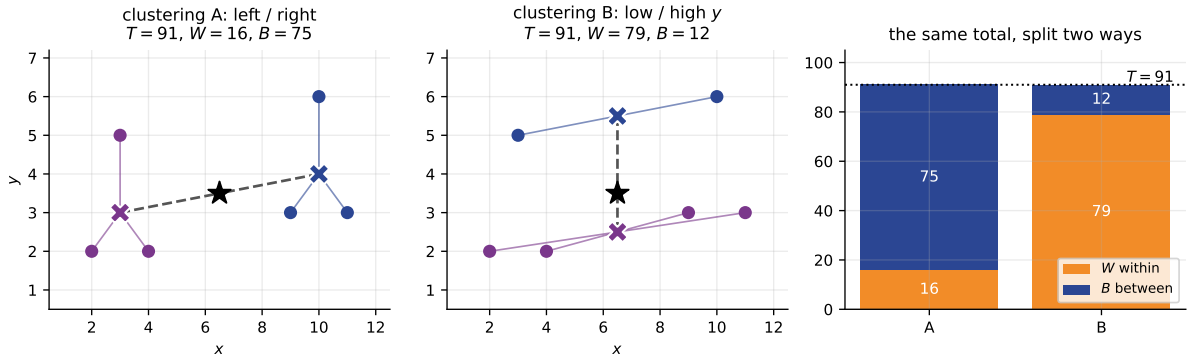


Figure 5: The six points under both clusterings. Coloured lines are the within-cluster displacements that make W ; dashed grey lines run from each centroid to the grand mean (the black star) and make B . Right: a fixed budget of $T = 91$, divided two ways.

One more check worth doing once. k-means with $k = 2$ on these six points recovers clustering A, and its `inertia_` attribute is exactly the W you computed:

```
k-means centroids (sorted by x)
(3.00, 3.00)
(10.00, 4.00)
k-means labels      [1 1 1 0 0 0]
our clustering A    [0 0 0 1 1 1]
k-means inertia_    16.0000
our W for clustering A 16.0000
```

Note that the labels are `[1 1 1 0 0 0]` where ours were `[0 0 0 1 1 1]`. Same partition, different names. Cluster identifiers carry no meaning; Section 6.6 makes that precise.

5.5 Verified on real data, at every k

```
for k in range(1, 9):
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    T, W, B = scatter_decomposition(X, km.labels_)
    print(k, T, W, B, abs(T - W - B))
```

k	T	W	B	T-W-B
1	681.370600	681.370600	0.000000	0.000e+00
2	681.370600	152.347952	529.022648	1.137e-13
3	681.370600	78.851441	602.519159	0.000e+00
4	681.370600	57.228473	624.142127	1.137e-13
5	681.370600	46.446182	634.924418	0.000e+00
6	681.370600	39.039987	642.330613	1.137e-13
7	681.370600	34.420192	646.950408	0.000e+00
8	681.370600	30.064593	651.306007	0.000e+00

$T = 681.370600$ in every row, and the largest residual is 1.137×10^{-13} — double-precision rounding, not mathematics. At $k = 1$ everything is within and nothing is between. As k grows the budget migrates from W into B .

5.6 Why W cannot choose k

“Pick the k with the smallest W ” returns $k = n$

W is monotone decreasing in k . Splitting any cluster into two can only decrease W (by equation (2), the two new centroids each fit their own members at least as well as the old one did), and at $k = n$ every point is its own centroid so $W = 0$ exactly. **Minimising W is not an objective, it is a countdown.** The same argument, mirrored, says B/T climbs monotonically to 1.

k	W	inertia_	B/T	CH
2	152.347952	152.347952	0.7764	513.9245
3	78.851441	78.851441	0.8843	561.6278
4	57.228473	57.228473	0.9160	530.7658
5	46.446182	46.446182	0.9318	495.5415
6	39.039987	39.039987	0.9427	473.8506
7	34.420192	34.420192	0.9495	447.9634
8	30.064593	30.064593	0.9559	439.4607

W falls at every step; it can never choose k on its own

The middle column is scikit-learn’s `inertia_`, which agrees with our W to every printed digit — `inertia_` is the within-cluster sum of squares, nothing more.

The traditional response is the **elbow method**: plot W against k and look for the point where the curve stops falling steeply. On iris that suggests $k = 3$, and here that happens to be a good answer. But be honest about what it is. It is a visual judgement on a curve that has no mathematical optimum, two people reading the same plot routinely name different elbows, and on many real datasets the curve is a smooth arc with no elbow at all. Use it to generate candidates, and then justify the choice with something else.

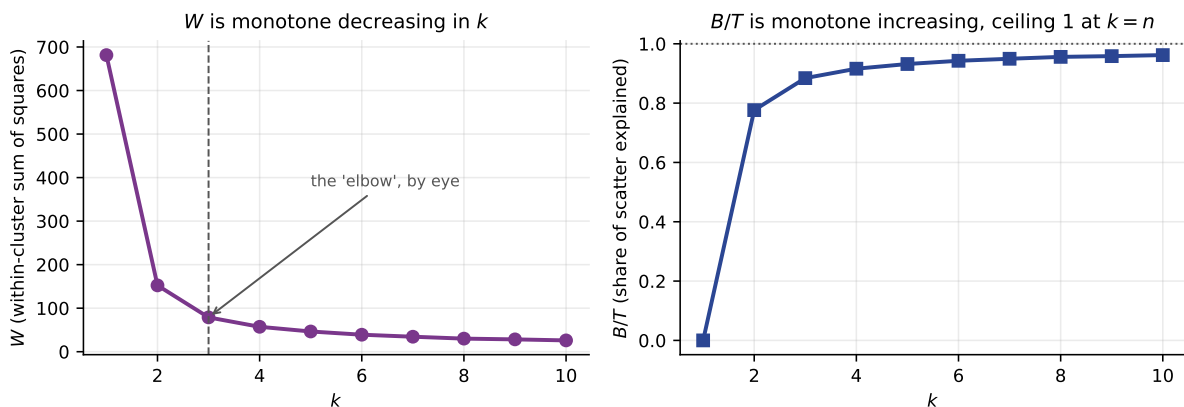


Figure 6: W falls monotonically towards zero and B/T climbs monotonically towards one. Neither curve has an interior optimum, so neither can select k by itself.

5.7 Calinski–Harabasz: the decomposition with the penalty put back

The fix is to charge for the clusters you use. The **Calinski–Harabasz index**, sometimes called the variance ratio criterion, divides each sum of squares by its degrees of freedom:

$$\text{CH} = \frac{B/(k-1)}{W/(n-k)}. \quad (6)$$

Higher is better. Adding a cluster increases B , but it also spends a degree of freedom, and past the useful point the numerator stops growing fast enough to pay for it. Unlike W , this index has an interior maximum.

CH on the six points

Clustering A: $B = 75$, $W = 16$, $k = 2$, $n = 6$, so

$$\text{CH}_A = \frac{75/(2-1)}{16/(6-2)} = \frac{75}{4} = \mathbf{18.75}.$$

Clustering B: $B = 12$, $W = 79$, so $\text{CH}_B = \frac{12}{79/4} = \frac{48}{79} = \mathbf{0.6076}$. Clustering A is about thirty times better by this measure.

```
A: left/right      B/T = 0.8242   CH by hand = 18.7500   sklearn = 18.7500
B: low/high y     B/T = 0.1319   CH by hand = 0.6076   sklearn = 0.6076
k=3  B/(k-1) = 301.259579  W/(n-k) = 0.536404  ratio = 561.6278
      sklearn calinski_harabasz_score = 561.6278
k=6  B/(k-1) = 128.466123  W/(n-k) = 0.271111  ratio = 473.8506
      sklearn calinski_harabasz_score = 473.8506
```

Try it yourself

Compute CH by hand for the iris clustering at $k = 2$: you have $B = 529.022648$, $W = 152.347952$, $n = 150$. You should get $\frac{529.022648}{152.347952/148} = 513.92$, matching the table in Section 3.1. If you get something near 3.47 you forgot the degrees of freedom.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Kriegel, Sander and Xu, “A density-based algorithm for discovering clusters (DBSCAN),” KDD 1996.

6 Internal and external validation

6.1 Two kinds of validation, and only one is usually available

Internal validation scores a clustering using nothing but the points and the assignments. No labels anywhere. This is what you actually have on a real problem, and it is what Sections 6.2 to 6.5 develop: the silhouette, Calinski–Harabasz, Davies–Bouldin.

External validation compares your clustering to a known grouping supplied from outside. It requires a label column you did not use for fitting, which on real clustering problems you do not have. We use it here because `load_iris` happens to ship with one — which lets us do something valuable: *check the internal measures against a known answer and find out where they are wrong*.

Do not use external validation as your objective

If you have labels for all your rows, you have a classification problem, not a clustering problem, and you should be training a classifier. External indices in these notes are a diagnostic instrument for studying internal indices, not a model-selection criterion you can use in the field.

6.2 The silhouette, by hand

For each point i in cluster C_j define

$$a(i) = \frac{1}{n_j - 1} \sum_{x \in C_j, x \neq i} d(i, x), \quad b(i) = \min_{l \neq j} \frac{1}{n_l} \sum_{x \in C_l} d(i, x), \quad s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}. \quad (7)$$

In words: $a(i)$ is the average distance to the point's own cluster-mates, $b(i)$ the average distance to the members of the *nearest other* cluster. If a is much smaller than b then s is near +1 and the point is comfortably placed. If they are equal s is 0 and the point is on the border. **If $a > b$ then s is negative and the point is on average closer to another cluster's members than to its own** — the clearest single signal that it has been misassigned. Singleton clusters are defined to have $s = 0$.

The **silhouette score** of a clustering is the mean of $s(i)$ over all points.

$s(P_1)$ under clustering A

The pairwise distances among the six points of Section 5.4:

	P1	P2	P3	P4	P5	P6
P1	0.0000	2.0000	3.1623	7.0711	9.0554	8.9443
P2	2.0000	0.0000	3.1623	5.0990	7.0711	7.2111
P3	3.1623	3.1623	0.0000	6.3246	8.2462	7.0711
P4	7.0711	5.0990	6.3246	0.0000	2.0000	3.1623
P5	9.0554	7.0711	8.2462	2.0000	0.0000	3.1623
P6	8.9443	7.2111	7.0711	3.1623	3.1623	0.0000

P_1 is in the cluster $\{P_1, P_2, P_3\}$, so

$$a(P_1) = \frac{d(P_1, P_2) + d(P_1, P_3)}{2} = \frac{2 + 3.162278}{2} = \frac{5.162278}{2} = 2.581139.$$

The only other cluster is $\{P_4, P_5, P_6\}$, so

$$b(P_1) = \frac{7.071068 + 9.055385 + 8.944272}{3} = \frac{25.070725}{3} = 8.356908.$$

Since $b > a$, the denominator is b , and

$$s(P_1) = \frac{8.356908 - 2.581139}{8.356908} = \frac{5.775769}{8.356908} = \mathbf{0.691137}.$$

```
from scipy.spatial.distance import cdist
from sklearn.metrics import silhouette_samples, silhouette_score
D = cdist(P6, P6)
s = silhouette_samples(P6, LAB_A)
for i in range(6):
    same = D[i][LAB_A == LAB_A[i]]
    a = same.sum() / (len(same) - 1)
    b = D[i][LAB_A != LAB_A[i]].mean()
    print(i + 1, a, b, (b - a) / max(a, b), s[i])
```

point	a(i)	b(i)	s(i)	sklearn
P1	2.581139	8.356908	0.691137	0.691137
P2	2.581139	6.460397	0.600467	0.600467
P3	3.162278	7.213945	0.561644	0.561644
P4	2.581139	6.164881	0.581316	0.581316
P5	2.581139	8.124221	0.682291	0.682291
P6	3.162278	7.742147	0.591550	0.591550
mean silhouette	0.618068		sklearn 0.618068	
clustering B	-0.071164		sklearn -0.071164	

Two things. Our hand arithmetic and `silhouette_samples` agree in every digit shown; the library is doing exactly equation (7) and nothing else. And clustering B scores **-0.071164** — negative, meaning the average point of that partition is nearer the other cluster's members than its own. The silhouette does not merely prefer A to B; it says B is actively wrong.

6.3 Silhouette on real data, and the profile

```
k=2 mean 0.6810 negative silhouettes: 0 of 150 min 0.0407
    cluster 0 n= 53 mean s = 0.7695
    cluster 1 n= 97 mean s = 0.6327
k=3 mean 0.5528 negative silhouettes: 0 of 150 min 0.0264
    cluster 0 n= 62 mean s = 0.4173
    cluster 1 n= 50 mean s = 0.7981
    cluster 2 n= 38 mean s = 0.4511
```

The per-cluster breakdown is more informative than the mean. At $k = 3$ the three clusters score 0.4173, **0.7981** and 0.4511: one crisp cluster — *setosa*, cluster 1, with 50 members — and two mediocre ones that are really one region cut in half. A mean of 0.5528 conceals that completely.

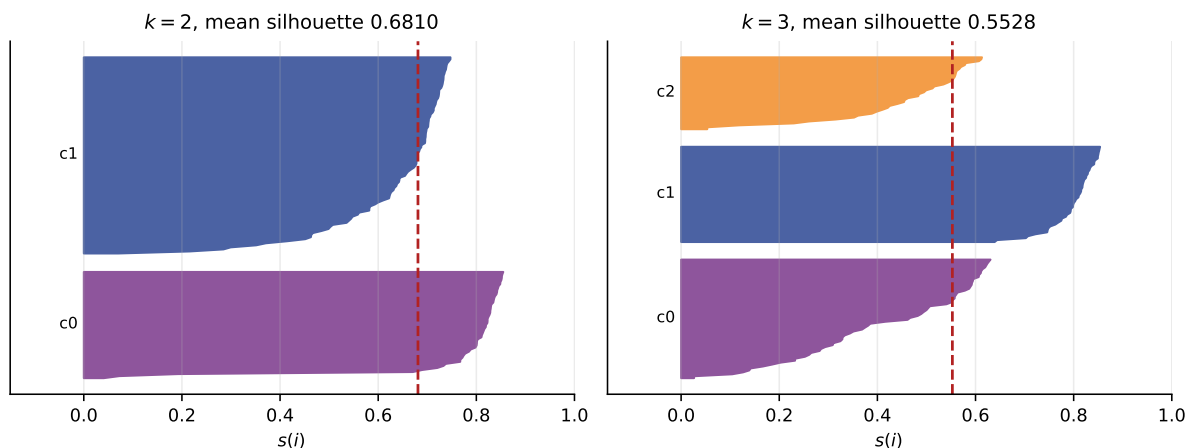


Figure 7: The silhouette profile: every point’s $s(i)$, sorted within its cluster, with the overall mean as a dashed line. A cluster whose bars fall well below the mean line is the one to investigate.

Key idea

Plot the profile, not just the average. A silhouette score of 0.55 can mean “three mediocre clusters” or “one excellent cluster and two poor ones”, and the action you should take is different in the two cases.

6.4 Davies–Bouldin

The third internal index in common use:

$$DB = \frac{1}{k} \sum_{j=1}^k \max_{l \neq j} \frac{S_j + S_l}{d(c_j, c_l)}, \quad S_j = \frac{1}{n_j} \sum_{x \in C_j} \|x - c_j\|. \quad (8)$$

S_j is the average distance from the members of a cluster to its own centroid — its spread. For each cluster you find its *worst* neighbour, the one whose combined spread is largest relative to the distance between the centroids, and you average those worst cases. **Lower is better**, and zero is unattainable.

DB on the six points, clustering A

Cluster 1 has centroid $(3, 3)$ and its members lie at distances $\sqrt{2} = 1.414214$, $\sqrt{2} = 1.414214$ and 2, so

$$S_1 = \frac{1.414214 + 1.414214 + 2}{3} = \frac{4.828427}{3} = 1.609476.$$

Cluster 2 is a translate of cluster 1, so $S_2 = 1.609476$ as well. The centroids are $(3, 3)$ and $(10, 4)$, a distance $\sqrt{50} = 7.071068$ apart. With only two clusters, each has exactly one neighbour, so

$$DB = \frac{1}{2} \left(\frac{S_1 + S_2}{7.071068} + \frac{S_2 + S_1}{7.071068} \right) = \frac{3.218951}{7.071068} = \mathbf{0.455228},$$

which is what `davies_bouldin_score` returns. Clustering B scores 2.358045 — more than five times worse.

The three indices measure related but genuinely different things. The silhouette works on **pair-wise distances**; Calinski–Harabasz works on the **scatter decomposition**; Davies–Bouldin works on **centroid spreads and centroid separations**. There is no reason for three different summaries of a partition to rank two partitions the same way, and they do not.

6.5 All three across k , with no labels used

```
for k in range(2, 9):
    lab = KMeans(n_clusters=k, n_init=10, random_state=0).fit_predict(X)
    print(k, silhouette_score(X, lab), calinski_harabasz_score(X, lab),
          davies_bouldin_score(X, lab))
```

k	silhouette	CH	DB	ARI	NMI
2	0.6810	513.92	0.4043	0.5399	0.6565
3	0.5528	561.63	0.6620	0.7302	0.7582
4	0.4981	530.77	0.7803	0.6498	0.7219
5	0.4887	495.54	0.8060	0.6079	0.6939
6	0.3648	473.85	0.9142	0.4475	0.6380
7	0.3569	447.96	0.9628	0.4813	0.6689
8	0.3597	439.46	0.9266	0.4570	0.6581

```
silhouette prefers k = 2 (higher is better)
Calinski-Harabasz k = 3 (higher is better)
Davies-Bouldin k = 2 (lower is better)
the labels prefer k = 3 (ARI, higher is better)
two of the three internal indices disagree with the labels
```

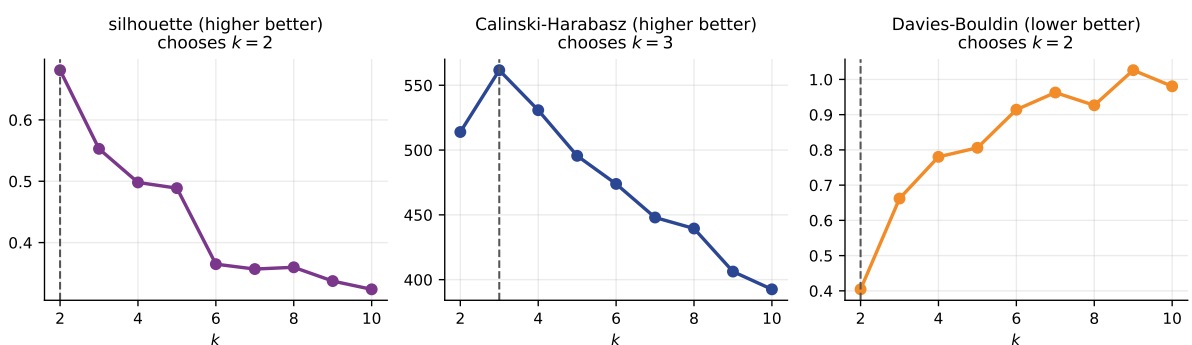


Figure 8: Three internal indices on iris, all computed with no access to the species column. They do not agree on k .

6.6 External validation: ARI and NMI

An external index compares two *partitions* of the same objects — your clustering and the known grouping. It must be invariant to how the clusters are named, because cluster 0 is not class 0; it is just the first group the algorithm happened to build.

```
lab3 = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(X)
```

```
perm = np.array([0: 2, 1: 0, 2: 1][v] for v in lab3) # rename only
print(adjusted_rand_score(y, lab3), adjusted_rand_score(y, perm))
```

```
relabelling the clusters 0->2, 1->0, 2->1 changes nothing:
ARI before 0.730238    after 0.730238
NMI before 0.758176    after 0.758176
cluster identifiers are names, not classes
```

`accuracy_score(y, clusters)` is meaningless

It compares label *values*, and cluster label values are arbitrary. The same partition can score 0.9 or 0.0 depending on which permutation the algorithm happened to return. Use ARI, NMI or an explicitly matched measure; never accuracy.

The **Rand index** counts the point *pairs* the two partitions agree about — pairs together in both, or apart in both — divided by $\binom{n}{2}$. That is permutation-invariant, but it has a serious defect.

```
rng = np.random.default_rng(0)
for _ in range(200):
    r = rng.integers(0, 3, len(y))
    ri.append(rand_score(y, r)); ari_.append(adjusted_rand_score(y, r))
```

```
200 completely random 3-way labellings of iris
Rand index          mean +0.5567    sd 0.0035
adjusted Rand index mean -0.0010    sd 0.0073
normalised mutual info mean +0.0114    sd 0.0067
```

A clustering produced by rolling a die scores **0.5567** on the Rand index. The reason is that most pairs of points are in *different* groups under both partitions, and the Rand index counts those as agreements. With $k = 3$ and roughly equal groups, about $2/3$ of pairs are apart under each partition by chance alone.

The **adjusted Rand index** fixes this by subtracting the expected value under a random labelling and rescaling so that the maximum is 1:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]} \quad (9)$$

It scores **-0.0010** on the same random labellings — zero, to within sampling noise — and it can go negative, which means “worse than chance”.

Normalised mutual information takes a different route. Treat the cluster assignment and the true label as two discrete random variables and compute their mutual information $I(U; V)$, then normalise by the entropies: $\text{NMI} = I(U; V) / \sqrt{H(U)H(V)}$ in scikit-learn’s default. It lies in $[0, 1]$ and is 1 only when the two partitions determine each other. Plain NMI is only weakly corrected for chance (0.0114 above, not 0); use `adjusted_mutual_info_score` if you need a fully corrected version.

6.7 The honest disagreement

State it plainly, because it is the most important honest result in the lecture: **on iris, the silhouette prefers a clustering that the labels do not.** $k = 2$ scores 0.6810 and merges two of the three species; $k = 3$ scores only 0.5528 and recovers them.

There are three defensible readings and you should be able to give all three.

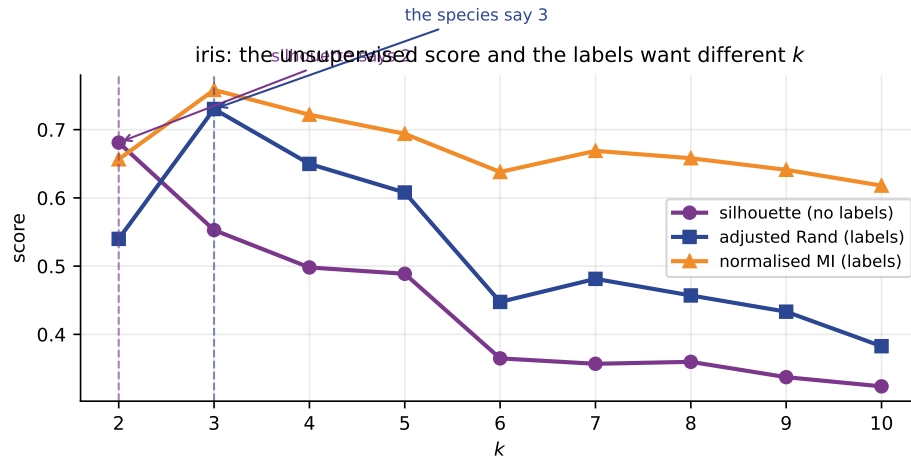


Figure 9: The unsupervised score and the labels want different k . The silhouette peaks at $k = 2$; the adjusted Rand index and the normalised mutual information both peak at $k = 3$.

1. *The index is right and the labels are the wrong question.* In these four measurements, *versicolor* and *virginica* overlap. A measure of separation should say so.
2. *The labels are right and the index is limited.* Species are a biological fact; petal geometry is a noisy proxy for it, and no unsupervised measure can recover a distinction the features barely encode.
3. *Both, and the useful output is the pair.* $k = 2$ is the strongest *geometric* structure; $k = 3$ is the strongest structure *that corresponds to something outside the data*. Reporting both, with the numbers, is more informative than picking one.

What you must not do is quote 0.6810, call the clustering validated, and move on. That number is the value of one index at one setting, and two other indices and the ground truth all have something different to say.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Kriegel, Sander and Xu, “A density-based algorithm for discovering clusters (DBSCAN),” KDD 1996.

7 Clustering as a preprocessing tool

Clustering is not only an end in itself. Two uses feed a supervised model, and both are worth measuring rather than assuming.

7.1 Cluster membership as an engineered feature

Fit k-means on the training rows. Then add, to each row, either the k indicator columns saying which cluster it fell into, or the k distances to the centroids. A **linear** model given those columns can fit a separate intercept in each cluster, which makes it piecewise — non-linear — for the price of k extra columns and no labels.

```
Xm, ym = make_moons(n_samples=600, noise=0.15, random_state=3)
Xtr, Xte, ytr, yte = train_test_split(Xm, ym, test_size=0.3,
                                     random_state=0, stratify=ym)
km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(Xtr)
Otr = np.eye(k)[km.labels_]; Ote = np.eye(k)[km.predict(Xte)]
LogisticRegression(max_iter=5000).fit(np.hstack([Xtr, Otr]), ytr)
```

```

logistic regression on the two raw columns    0.9056
  k      + one-hot      + distances
  2      0.9444         0.9667
  4      0.9722         0.9167
  6      0.9889         0.9778
  8      0.9944         0.9833
 10      0.9944         0.9833
 15      0.9944         0.9889
 20      0.9889         0.9889

```

From 0.9056 to **0.9944**: a model that could only draw a straight line now follows a curved boundary, and the eight regions it uses were built without looking at a single label.



Figure 10: Left: k-means with $k = 8$ on the training rows, labels hidden. Middle: logistic regression on the two coordinates draws a line. Right: the same model with eight cluster indicator columns added follows the cluster edges.

7.2 And the null result, reported as it happened

The same recipe on `load_breast_cancer`:

```

logistic regression on the 30 scaled columns    0.9591
+ 2 cluster distances                          0.9532
+ 5 cluster distances                          0.9591
+ 10 cluster distances                         0.9649
+ 20 cluster distances                         0.9591
report the null result: on this table it buys essentially nothing

```

The best of the four settings gains +0.0058, which on 171 test rows is **one patient**. Two of the four made things worse. This is not a result; it is noise, and reporting it as a 0.6 percentage point improvement would be dishonest.

Why the difference? The trick pays when the decision boundary is non-linear *and* the clusters happen to line up with it. On the moons both hold. On breast cancer the thirty standardised columns are already close to linearly separable, so there is no curvature for the extra columns to supply.

Key idea

“Add cluster features” is a hypothesis about your data, not a technique that always helps. Measure the baseline, measure the augmented model, and report whichever way it came out.

7.3 Cluster-then-label: spending a labelling budget

Rows are usually cheap and labels are usually expensive — a radiologist’s time, a lawyer’s reading, a chemist’s assay. Suppose you can afford 50 labels out of 1257 handwritten digits.

How should you choose which 50?

The naive answer is to pick 50 rows at random. The better answer uses clustering: run k-means with $k = 50$, and label the one row *closest to each centroid*. Those 50 rows are, by construction, spread across the whole data and each is typical of its neighbourhood. Then go one step further and **propagate** each label to every row in its cluster, which turns 50 labels into 1257 approximate ones.

```
km = KMeans(n_clusters=B, n_init=10, random_state=0).fit(Xtr)
rep = np.argmax(km.transform(Xtr), axis=0) # nearest row to each centroid
yprop = np.empty(len(Xtr), dtype=int)
for i in range(B): yprop[km.labels_ == i] = ytr[rep[i]]
LogisticRegression(max_iter=5000).fit(Xtr, yprop)
```

fully supervised baseline (1257 labels) 0.9722				
budget	random	sd	representative	propagated
10	0.4744	0.0180	0.7926	0.7741
20	0.6378	0.0337	0.8463	0.9000
30	0.7219	0.0374	0.9056	0.9111
50	0.8104	0.0274	0.8981	0.9389
100	0.8907	0.0184	0.9370	0.9370

At a budget of 50: random selection gives 0.8104, representative selection 0.8981, and propagation **0.9389** — against 0.9722 for labelling everything. **Ninety-six per cent of the achievable accuracy for four per cent of the labelling cost.** At a budget of 10 the gap is even starker: 0.4744 against 0.7926.

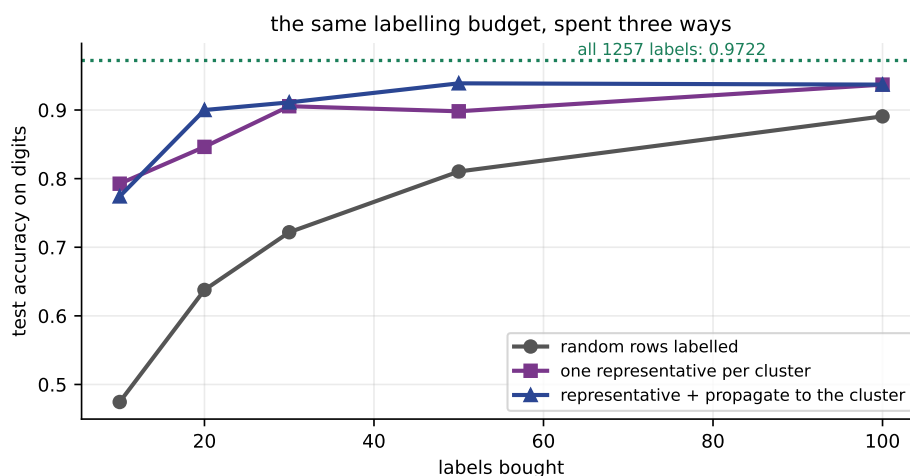


Figure 11: The same labelling budget spent three ways on `load_digits`. Random selection is the grey line; the dotted line is what all 1257 labels achieve.

Propagation clearly introduces errors, so it is worth measuring how many.

```
50 labels bought; 1257 training rows labelled by propagation
propagated labels that are actually correct: 1206 of 1257 (0.9594)
so the cheap model trains on 4.06% wrong labels
```

Fifty-one of the 1257 propagated labels are wrong. The model still wins, because logistic regression tolerates 4% label noise far better than it tolerates having only 50 training rows. That trade — a little noise for a lot of data — is the whole idea.

7.4 The leakage rule

Fit the clusterer on the training rows only

Both techniques above fit k-means on `Xtr` and then call `predict` on `Xte`. If you fit k-means on the whole dataset before splitting, the centroids — and therefore the engineered features — have been computed using the test rows, and your test score is optimistic.

This is the same rule you met for `StandardScaler`, in a new costume. The general form is: **any object with a fit method must be fit inside the training fold**. Inside a `Pipeline` and `cross_val_score`, scikit-learn enforces this for you, which is the main reason to use them.

There is a related trap worth naming. Clustering is sometimes done after dimensionality reduction, and the order matters less than people fear but not zero:

```
PCA first two components explain 0.9777 of the variance
cluster in 4D then project : ARI vs labels 0.7302
project to 2D then cluster : ARI vs labels 0.7163
the two clusterings agree with each other : ARI 0.9803
```

On iris the two orders agree at ARI 0.9803 and differ by 0.0139 against the species — because two components already carry 97.77% of the variance. On a dataset where they carry 60%, the two orders can differ a great deal. Decide deliberately, and say which you did.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Krieger, Sander and Xu, “A density-based algorithm for discovering clusters (DBSCAN),” KDD 1996.

8 General applications of clustering

Four families of use, kept concrete.

Segmentation. Customers grouped by purchase behaviour, so a campaign can be written for each group instead of for an average customer nobody resembles. Students grouped by attempt patterns, so that the right intervention reaches the right group. The cluster here corresponds to *a population you will treat differently*, and the test of the clustering is whether treating them differently works.

Organising a corpus. Documents grouped into topics for browsing; search results grouped so that a query with two meanings returns two piles; genes grouped by correlated expression profiles, which is how co-regulated gene sets were first found. The cluster corresponds to *a heading in a table of contents that nobody wrote*.

Compression and summarisation. Vector quantisation: replace every value by its cluster centre, and store the small codebook plus one index per item. Colour palettes are the textbook case.

```
image 427 x 640 pixels, 96615 distinct colours
k= 2  palette  2 colours  MSE 0.019764  storage  4.17% of 24-bit
k= 4  palette  4 colours  MSE 0.007028  storage  8.34% of 24-bit
k= 8  palette  8 colours  MSE 0.003246  storage 12.51% of 24-bit
k= 16 palette 16 colours  MSE 0.001792  storage 16.69% of 24-bit
k= 32 palette 32 colours  MSE 0.000998  storage 20.88% of 24-bit
```

Sixteen colours in place of 96 615 reproduces the image at a mean squared error of 0.0018 per channel, in about a sixth of the storage. The cluster corresponds to *an entry in a codebook*.

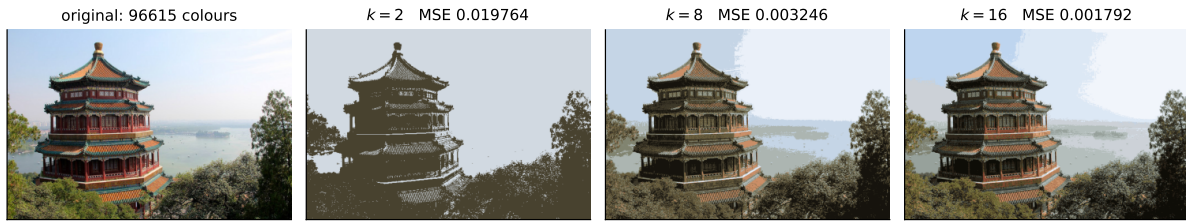


Figure 12: Colour quantisation. Each pixel is a point in $\mathbb{R}^3$; k-means finds a palette; every pixel is replaced by its centroid.

Anomaly detection. Points that fall in no cluster, or in a cluster of size one, are candidates for fraudulent transactions, network intrusions or faulty sensors. Density-based methods hand you this for free, since they already refuse to assign some points. Measured on the moons with twelve uniform outliers planted:

```
rng = np.random.default_rng(0) # the one seed for this lecture
Xm, _ = make_moons(n_samples=500, noise=0.06, random_state=7)
out = rng.uniform([-1.5, -1.0], [2.5, 1.5], size=(12, 2))
lab = DBSCAN(eps=0.20, min_samples=5).fit_predict(np.vstack([Xm, out]))
```

```
500 structured points + 12 uniform outliers
DBSCAN flagged 8 points as noise
of those, 8 are planted outliers and 0 are structured points
recall on the planted outliers 0.6667, precision 1.0000
```

Report that as it is: **precision 1.0000, recall 0.6667**. Every point DBSCAN flagged really was an outlier, but it missed a third of them, because four of the twelve landed inside or against the crescents where the local density is perfectly ordinary. That is the honest limit of the technique: *it detects points in sparse regions, which is not the same thing as points that do not belong*.

Finally, a use that is really a demonstration. Run k-means with $k = 10$ on 1797 digit images with the labels hidden:

```
k-means with k=10 on 1797 unlabelled digit images
ARI vs the true digit 0.6657, NMI 0.7425
cluster : dominant digit (purity)
  0 : 0 (176 of 178 = 0.99)      5 : 6 (177 of 182 = 0.97)
  1 : 1 (100 of 223 = 0.45)      6 : 4 (165 of 169 = 0.98)
  2 : 7 (174 of 208 = 0.84)      7 : 5 (136 of 150 = 0.91)
  3 : 1 ( 54 of  87 = 0.62)      8 : 9 (139 of 247 = 0.56)
  4 : 3 (154 of 178 = 0.87)      9 : 2 (148 of 175 = 0.85)
overall purity 0.7919
```

Knowing nothing but 64 pixel values, the algorithm produces groups that are 79% pure. Zeros, sixes and fours come out almost perfectly. The failures are the interesting part: **ones are split across two clusters** (0.45 and 0.62 pure), and nines and eights are muddled — which is roughly what a human misreads too.

Purity is not an honest index on its own

Purity always increases as k increases, and reaches 1 at $k = n$. It is readable and it is useful alongside a confusion table, but it is in the same family as W : monotone in k , and therefore useless for choosing k . Quote ARI or NMI beside it.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Kriegel, Sander and Xu, “A density-based algorithm for discovering clusters (DBSCAN),” KDD 1996.

9 Summary

1. Clustering divides objects into groups so that within-group objects are similar and between-group objects are not. There is **no target column**, so the objective is a modelling choice and validation must be rebuilt from geometry.
2. **There is no single right answer.** On iris the silhouette prefers $k = 2$ (0.6810 against 0.5528) while the species prefer $k = 3$ (ARI 0.7302 against 0.5399); the two clusterings agree with each other at ARI only **0.5516**.
3. Feature scaling *is* the definition of similarity. On wine, raw against standardised gives ARI to the cultivar of 0.3711 and **0.8975**, and the two clusterings agree at 0.3543 — because **proline** carries 99.77% of the raw variance.
4. Five families: **partitioning, hierarchical, density-based, grid-based, model-based**, distinguished by what they assume a cluster looks like.
5. Those assumptions have consequences. On two moons k-means scores **0.2445** and DBSCAN **1.0000**; on elongated blobs k-means scores 0.5554 and a Gaussian mixture 1.0000; on unequal spread single linkage scores **0.0002**. **No algorithm wins all four datasets.**
6. A grid clusterer is a histogram plus a flood fill: 43 dense cells of 100, two components, ARI 0.9643 — and at $\tau = 2$, $G = 12$ the arms bridge and it collapses to **0.0000**.
7. Centroid $c_j = \frac{1}{n_j} \sum_{x \in C_j} x$; it minimises the sum of squared distances to its members, and that single fact proves everything else.
8. $T = W + B$. On six points, $91 = 16 + 75$ for one clustering and $91 = 79 + 12$ for another. On iris, $T = 681.370600$ at every k with residual at most 1.14×10^{-13} .
9. Since T is fixed, **minimising W and maximising B are one problem**. W falls monotonically to 0 at $k = n$, so it cannot choose k ; the elbow is a visual heuristic on a curve with no optimum.
10. $CH = \frac{B/(k-1)}{W/(n-k)}$ restores the penalty: $75/4 = \mathbf{18.75}$ by hand, 18.7500 from scikit-learn.
11. Silhouette $s(i) = (b - a) / \max(a, b)$. By hand $s(P_1) = \mathbf{0.691137}$; the mean over the six is 0.618068, and the rival clustering scores **-0.071164** — negative means misassigned. Davies–Bouldin on the same clustering is 0.455228 by hand.
12. Internal indices disagree with each other: on iris, silhouette says $k = 2$, Calinski–Harabasz says $k = 3$, Davies–Bouldin says $k = 2$. **Report the ones that disagree with you.**
13. External measures must be chance-corrected: a random 3-way labelling of iris scores Rand **0.5567** but ARI **-0.0010**. Renaming clusters changes neither ARI nor NMI, so plain accuracy against cluster labels is undefined.
14. As preprocessing it works when it works: moons $0.9056 \rightarrow \mathbf{0.9944}$ with eight cluster indicators, but **breast_cancer** $0.9591 \rightarrow 0.9649$, which is one patient.
15. Cluster-then-label reaches **0.9389** on 50 labels against 0.8104 spent at random and 0.9722 for all 1257, at the cost of 4.06% wrong propagated labels.
16. Applications: segmentation, corpus organisation, vector quantisation (96 615 colours to 16 at MSE 0.0018), and anomaly detection (precision 1.0000, recall 0.6667 on planted outliers).

10 Practice questions

Work these with a pen before looking at Section 11. Questions 1, 2 and 5 are pure hand arithmetic and are the kind most likely to appear in a written paper.

1. **(Centroids, W , B , T by hand.)** Take the six points $P_1(2, 2)$, $P_2(4, 2)$, $P_3(3, 5)$, $P_4(9, 3)$, $P_5(11, 3)$, $P_6(10, 6)$ and the clustering $\mathcal{C} = \{\{P_1, P_2, P_3, P_4\}, \{P_5, P_6\}\}$. Compute both centroids, then W , B and T , and verify $T = W + B$. Compare W with the value 16 obtained for clustering A in Section 5.4 and say which clustering k-means would prefer.
2. **(Silhouette by hand, including a negative one.)** Using the distance matrix printed in Section 6.2 and the clustering $\mathcal{C}$ of Question 1, compute $a(P_4)$, $b(P_4)$ and $s(P_4)$. Interpret the sign of your answer in one sentence.
3. **(Calinski–Harabasz.)** For the clustering $\mathcal{C}$ of Question 1, compute CH. Then explain, in two sentences, why dividing B by $k - 1$ prevents the index from being maximised at $k = n$.
4. **(Why W is monotone.)** Prove that splitting one cluster into two non-empty parts cannot increase W . You may use equation (2). What does your proof imply about “choose the k that minimises W ”?
5. **(Davies–Bouldin by hand.)** Two clusters have centroids $(0, 0)$ and $(6, 8)$, with average member-to-centroid distances $S_1 = 1.5$ and $S_2 = 2.5$. Compute the Davies–Bouldin index. If a third cluster were added at $(3, 4)$ with $S_3 = 3.0$, would you expect DB to rise or fall? Why?
6. **(Chance correction.)** A colleague reports that their clustering of 150 items into 3 groups achieves a Rand index of 0.56 against a known grouping, and describes this as “moderate agreement”. What is wrong with the claim, what number should they have reported, and roughly what would it be?
7. **(Choosing a family.)** For each of the following, name the family of clustering algorithm you would try first and give the shape assumption that justifies it. (a) Customers described by ten standardised spending features, expected to form a few broad groups of similar size. (b) GPS pings from delivery riders, where the interesting structure is roads and there is a lot of stray noise. (c) A dataset where you believe each group is an elongated ellipse at a different angle. (d) Two billion records, where you can afford one pass over the data.
8. **(Scaling.)** On wine, clustering the raw table gives ARI 0.3711 against the cultivar and clustering the standardised table gives 0.8975. A student concludes “therefore you should always standardise before clustering”. Give one situation where that advice is wrong, and state the general principle it should be replaced by.
9. **(Cluster-then-label.)** With a budget of 50 labels on `load_digits`, propagation reached 0.9389 while training only on the 50 representatives reached 0.8981 — even though propagation introduces 51 wrong labels. Explain the mechanism. Then say what you would expect to happen to the propagation result if the 50 clusters were replaced by 5.
10. **(Leakage and reporting.)** A report says: “We ran k-means on the full dataset, added the cluster identity as a feature, split the data 70/30, and improved test accuracy from 0.951 to 0.958.” Identify two separate problems with this sentence, and say what you would need to see before believing the improvement.

11 Solutions

Solution 1 — centroids, W , B , T

Centroids. For $\{P_1, P_2, P_3, P_4\}$:

$$c_1 = \left(\frac{2+4+3+9}{4}, \frac{2+2+5+3}{4} \right) = \left(\frac{18}{4}, \frac{12}{4} \right) = (4.5, 3.0).$$

For $\{P_5, P_6\}$:

$$c_2 = \left(\frac{11+10}{2}, \frac{3+6}{2} \right) = (10.5, 4.5).$$

Within. Offsets from $c_1 = (4.5, 3)$:

point	offset	squared length
$P_1(2, 2)$	$(-2.5, -1.0)$	$6.25 + 1.00 = 7.25$
$P_2(4, 2)$	$(-0.5, -1.0)$	$0.25 + 1.00 = 1.25$
$P_3(3, 5)$	$(-1.5, +2.0)$	$2.25 + 4.00 = 6.25$
$P_4(9, 3)$	$(+4.5, 0.0)$	$20.25 + 0.00 = 20.25$

so the first cluster contributes $7.25 + 1.25 + 6.25 + 20.25 = 35$. Offsets from $c_2 = (10.5, 4.5)$ are $(0.5, -1.5)$ and $(-0.5, 1.5)$, each of squared length $0.25 + 2.25 = 2.5$, contributing 5. Hence

$$W = 35 + 5 = \mathbf{40}.$$

Between. The grand mean is $(6.5, 3.5)$.

$$\|(4.5, 3.0) - (6.5, 3.5)\|^2 = 4 + 0.25 = 4.25, \quad \|(10.5, 4.5) - (6.5, 3.5)\|^2 = 16 + 1 = 17,$$

$$B = 4(4.25) + 2(17) = 17 + 34 = \mathbf{51}.$$

Total. $T = 91$, unchanged — it depends only on the points and the grand mean, both of which are the same as in Section 5.4. And $40 + 51 = 91$. ✓

Which does k-means prefer? k-means minimises W , and $16 < 40$, so it prefers clustering A. Running it confirms this: it returns clustering A with `inertia_` = 16.0000.

```
C centroids [4.5 3. ] [10.5 4.5]
C: T 91.0000 W 40.0000 B 51.0000 sum 91.0000
```

Solution 2 — silhouette of P_4 under $\mathcal{C}$

P_4 now sits in the *first* cluster, $\{P_1, P_2, P_3, P_4\}$, so

$$a(P_4) = \frac{d(P_4, P_1) + d(P_4, P_2) + d(P_4, P_3)}{3} = \frac{7.0711 + 5.0990 + 6.3246}{3} = \frac{18.4947}{3} = 6.164881.$$

The only other cluster is $\{P_5, P_6\}$, so

$$b(P_4) = \frac{d(P_4, P_5) + d(P_4, P_6)}{2} = \frac{2.0000 + 3.1623}{2} = \frac{5.1623}{2} = 2.581139.$$

Here $a > b$, so the denominator is a :

$$s(P_4) = \frac{b - a}{\max\{a, b\}} = \frac{2.581139 - 6.164881}{6.164881} = \frac{-3.583742}{6.164881} = \mathbf{-0.581316}.$$

Interpretation. The value is negative, which means P_4 is on average much closer to the members of the other cluster than to its own cluster-mates — it has been put in the wrong group, and moving it to $\{P_5, P_6\}$ would improve the clustering. That is exactly right: $P_4(9, 3)$ belongs geometrically with P_5 and P_6 on the right, not with the three points on the left.

C per-point	[0.546904	0.521021	0.449462	-0.581316	0.52037	0.520662]
C silhouette	0.329517					

Solution 3 — Calinski–Harabasz

With $B = 51$, $W = 40$, $k = 2$, $n = 6$:

$$\text{CH} = \frac{51/(2-1)}{40/(6-2)} = \frac{51}{10} = \mathbf{5.10},$$

against 18.75 for clustering A — so CH agrees with W and with the silhouette that A is much better.

Why the $k-1$ matters. As k grows towards n , B grows towards T and W shrinks towards 0, so the raw ratio B/W diverges and would always select $k = n$. Dividing B by $k-1$ charges a growing price for each additional cluster, and dividing W by $n-k$ inflates the denominator’s per-degree-of-freedom size as k grows; together they turn a monotone quantity into one with an interior maximum.

Solution 4 — W is monotone in k

Let cluster C with centroid c be split into non-empty parts C' and C'' with centroids c' and c'' . Its contribution to W before the split is $\sum_{x \in C} \|x - c\|^2$. Apply equation (2) separately to each part, taking $m = c$:

$$\sum_{x \in C'} \|x - c\|^2 = \sum_{x \in C'} \|x - c'\|^2 + |C'| \|c' - c\|^2 \geq \sum_{x \in C'} \|x - c'\|^2,$$

and likewise for C'' . Adding the two and noting that the left sides sum to the pre-split contribution:

$$\sum_{x \in C} \|x - c\|^2 \geq \sum_{x \in C'} \|x - c'\|^2 + \sum_{x \in C''} \|x - c''\|^2,$$

which is the post-split contribution. Every other cluster is untouched, so W cannot increase. Equality holds only if $c' = c'' = c$. $\square$

Implication. “Choose the k minimising W ” always returns $k = n$, where every point is its own cluster and $W = 0$. W is a measure of fit, not a model-selection criterion; any procedure that selects k must add a penalty for k (as Calinski–Harabasz does) or use a different quantity entirely (as the silhouette does).

Solution 5 — Davies–Bouldin

The centroid distance is $d = \sqrt{6^2 + 8^2} = \sqrt{100} = 10$. With two clusters each has exactly one candidate neighbour, so the maximum in equation (8) is over a single term for each:

$$R_{12} = R_{21} = \frac{S_1 + S_2}{d} = \frac{1.5 + 2.5}{10} = 0.4, \quad \text{DB} = \frac{1}{2}(0.4 + 0.4) = \mathbf{0.40}.$$

Adding a third cluster at (3, 4). DB would *rise* (get worse). That point is the midpoint of the segment joining the two centroids, at distance 5 from each, and it has the largest spread of the three ($S_3 = 3.0$). Its ratios against both neighbours are $(1.5 + 3.0)/5 = 0.90$ and $(2.5 + 3.0)/5 = 1.10$, so cluster 3 contributes $\max = 1.10$; clusters 1 and 2 now also have a close, wide neighbour and their worst-case ratios rise to 0.90 and 1.10. The average is $(0.90 + 1.10 + 1.10)/3 = 1.0\bar{3}$, well above 0.40. The general lesson: DB punishes a cluster that is both wide and close to another one, and inserting a wide cluster between two tight ones is the worst case for it.

Solution 6 — chance correction

What is wrong. The Rand index is not corrected for chance. On 150 items in 3 groups, a completely random labelling scores about 0.5567 on average — so 0.56 is what you get from rolling a die, and it is evidence of nothing whatever. Describing it as “moderate agreement” is describing noise.

What to report. The **adjusted** Rand index, which subtracts the expected value under random labelling and rescales so that the maximum is 1: $ARI = (RI - \mathbb{E}[RI]) / (\max RI - \mathbb{E}[RI])$.

Roughly what value? Since the reported RI is essentially the chance value, the ARI would be approximately 0 — the measured mean over 200 random labellings of iris was -0.0010 with standard deviation 0.0073, so anything in roughly $[-0.02, +0.02]$ is indistinguishable from chance. Normalised mutual information would likewise be near 0.01, not 0.5.

Solution 7 — choosing a family

(a) **Partitioning (k-means).** The features are standardised, the groups are expected to be broad and of similar size, and no unusual shape is suggested. That is precisely the convex, roughly spherical, roughly equal-sized assumption k-means makes, and it is fast. Start there and check the silhouette profile.

(b) **Density-based (DBSCAN).** Roads are long thin connected structures — exactly the “dense region of arbitrary shape” that k-means cannot represent and DBSCAN can. It also has a built-in noise category, so the stray pings are handled rather than forced into a cluster. This is the two-moons result in a real setting.

(c) **Model-based (a full-covariance Gaussian mixture).** An elongated ellipse at an arbitrary angle is exactly a Gaussian with a full covariance matrix. This is the elongated-blobs row of Section 4.2, where the mixture scored 1.0000 and k-means 0.5554.

(d) **Grid-based.** The cost of a grid method is driven by the number of cells rather than the number of points, and it needs only one pass to build the histogram. No distance-matrix method is available at two billion records, and even k-means needs several passes.

Solution 8 — when not to standardise

A situation where the advice is wrong. When the columns are already in the same, meaningful unit and their relative magnitudes carry the information. Pixel intensities in an image, the 64 columns of `load_digits`, or the RGB values in the colour-quantisation example are all on a common scale, and standardising them column by column would inflate the low-variance pixels — mostly border pixels that are almost always blank — into important dimensions. A second case: when you deliberately want a high-variance column to dominate because it is the quantity you care about.

The principle it should be replaced by. Scaling is not a hygiene step, it is **the specification of what “similar” means**. Choose it from the meaning of the columns: standardise when the columns are in incomparable units and you want each to count equally; leave them alone when they share a unit and their relative sizes are meaningful; and in either case say which you did, because it is part of the result.

Solution 9 — why propagation wins

The mechanism. Training on the 50 representatives gives a model 50 perfectly labelled rows. Propagation gives it 1257 rows of which 1206 are correctly labelled — 95.94% accurate, so 51 are wrong. The model therefore trades a 4.06% label-noise rate for a 25× increase in training

data. For logistic regression on 64 features, 50 rows is severely data-starved: the estimate of each coefficient is dominated by sampling variance. Adding a thousand more rows reduces that variance far more than 4% label noise increases the bias, so the propagated model wins, 0.9389 against 0.8981.

With 5 clusters instead of 50. It would get much worse. Each cluster would then contain roughly 250 images spanning several digits, and a single representative's label would be propagated to all of them, so the propagated label accuracy would collapse from 95.94% towards something near $\frac{1}{5}$ of the rows. You would be training on mostly wrong labels, and the noise term would dominate. The budget-10 row of the table already shows the beginning of this effect: at $B = 10$, propagation (0.7741) is *worse* than the 10 representatives alone (0.7926) — the only budget in the table where that happens.

Solution 10 — leakage and reporting

Problem 1: leakage. k-means was fitted on the *full* dataset before the split, so the centroids were computed using the test rows. The engineered feature therefore encodes information about the test set, and the reported test accuracy is optimistically biased. The fix is to split first, fit k-means inside the training fold, and **predict** cluster identity for the test rows — or simply to put the clusterer and the classifier in a Pipeline and let `cross_val_score` enforce it.

Problem 2: the effect size is not distinguishable from noise. $0.951 \rightarrow 0.958$ on a single 30% test split is a difference of 0.007. If the test set has a few hundred rows that is one or two rows, well inside the variation you would see from changing `random_state`. This is the `breast_cancer` result of Section 7 exactly: +0.0058, which was one patient, and which the notes reported as a null result.

What you would need to see. Repeated cross-validation — say five repeats of five-fold — giving a mean and a standard deviation for both the baseline and the augmented model, with the clusterer inside the pipeline; the number of test rows; how k was chosen, and whether it was chosen on the test set (a third leak if so); and ideally a paired test across folds rather than two point estimates.

12 References and further reading

All links were checked on 24 August 2026. Free resources are marked [free].

Textbooks

- Han, Kamber and Pei, *Data Mining: Concepts and Techniques*, 3rd edition, Morgan Kaufmann, 2012. **The prescribed text for this lecture.** §10.1 is this lecture almost exactly — §10.1.1 “What is cluster analysis?” and its application list, §10.1.2 “Requirements for cluster analysis” (the checklist of scalability, arbitrary shape, noise tolerance and interpretability that Section 4 makes concrete), and §10.1.3 “An overview of basic clustering methods”, which is the five-family taxonomy in the same order used here. §10.2 is partitioning, §10.3 hierarchical, §10.4 density-based, §10.5 grid-based and §10.6 evaluation of clustering, including the chance-correction discussion of Section 6.6.
- Müller and Guido, *Introduction to Machine Learning with Python*, O'Reilly, 2016. **Also prescribed for this lecture.** §3.5 unsupervised learning: clustering — §3.5.1 k-means with the failure cases of Section 4.2 drawn, §3.5.2 agglomerative clustering, §3.5.3 DBSCAN, and **§3.5.4 “Comparing and evaluating clustering algorithms”**, which is Section 6 of these notes: the adjusted Rand index, why accuracy must not be used, and silhouette

scoring with its limitation on non-convex clusters. §3.5.5 is the summary. Notebooks [free]: https://github.com/amueller/introduction_to_ml_with_python

- James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*. [free] Chapter 12 is unsupervised learning: §12.1 states the “no simple goal” problem that Section 3 demonstrates, §12.4 is clustering, §12.4.1 k-means with the within-cluster variation objective written exactly as equation (4), §12.4.2 hierarchical clustering and the linkage comparison, and §12.4.3 “Practical issues in clustering” — the scaling decision of Section 3.2 and the warning that small perturbations change the answer. <https://www.statlearning.com/>
- Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, 2e, Springer. [free] §14.3 cluster analysis: §14.3.2 dissimilarities, §14.3.5 k-means with the $T = W + B$ decomposition stated at §14.3.4 in the notation used here, §14.3.8 hierarchical clustering, §14.3.11 the practical difficulty of choosing k . <https://hastie.su.domains/ElemStatLearn/>
- Kaufman and Rousseeuw, *Finding Groups in Data: An Introduction to Cluster Analysis*, Wiley, 1990. The book that introduced the silhouette and k-medoids. Chapter 2 is the silhouette in full, including the interpretation table for the average silhouette width that most subsequent textbooks copy.

Online courses

- **NPTEL, Introduction to Machine Learning** — Prof. Balaraman Ravindran, IIT Madras; the reference course for the semester. Its clustering week covers partitional and hierarchical clustering in the notation used here. [free] <https://nptel.ac.in/courses/106106139>
- **NPTEL, Data Mining** — Prof. Pabitra Mitra, IIT Kharagpur. Follows the Han–Kamber–Pei chapter structure, so its clustering weeks map directly onto §10.1–§10.5. [free] <https://nptel.ac.in/courses/106105174>
- **NPTEL, Introduction to Machine Learning** — Prof. Sudeshna Sarkar, IIT Kharagpur. A second treatment of clustering worked at the board, if the first does not land. [free] <https://nptel.ac.in/courses/106105152>
- **SWAYAM** — the national portal hosting NPTEL, with certification cycles. [free] <https://swayam.gov.in/>
- **Google, Introduction to Clustering** — a short course whose structure is close to this lecture: “What is clustering?”, “Clustering algorithms” (the taxonomy), “Clustering workflow”, and “Evaluating results”. [free] <https://developers.google.com/machine-learning/clustering>
- **Google Machine Learning Crash Course** — the background for the supervised half of Section 7. [free] <https://developers.google.com/machine-learning/crash-course>

Videos

- StatQuest with Josh Starmer, *StatQuest: K-means clustering* — the assign-and-update loop drawn out at a whiteboard, with the elbow method at the end. Watch this one first. [free] <https://youtu.be/4b5d3muPQmA>
- StatQuest with Josh Starmer, *StatQuest: Hierarchical Clustering* — the dendrogram and the linkage choices, which is the second family in Section 4 and the subject of a later lecture.

[free] <https://youtu.be/7xHsRkOdVwo>

- StatQuest with Josh Starmer, *Clustering with DBSCAN, Clearly Explained!!!* — core points, border points and noise, and why this handles the two-moons problem that k-means cannot. [free] <https://youtu.be/RDZUdRSD0ok>
- Victor Lavrenko, *K-means clustering: how it works* — a slower, more mathematical treatment including the objective function and the proof that the algorithm decreases it at every step. [free] https://youtu.be/_aWzGGNrcic

Documentation and interactive explainers

- scikit-learn user guide, 2.3. *Clustering* — the reference for every estimator used here. The overview table at the top lists each algorithm’s parameters, scalability and “**geometry (metric used)**”, which is the taxonomy of Section 4 in the documentation’s own words. §2.3.11 “Clustering performance evaluation” defines the adjusted Rand index, mutual-information scores, silhouette, Calinski–Harabasz and Davies–Bouldin, each with the chance-correction discussion. [free] <https://scikit-learn.org/stable/modules/clustering.html>
- scikit-learn example, *Comparing different clustering algorithms on toy datasets* — the same experiment as Section 4.2, extended to ten algorithms and six datasets, with run times. [free] https://scikit-learn.org/stable/auto_examples/cluster/plot_cluster_comparison.html
- scikit-learn example, *Selecting the number of clusters with silhouette analysis on KMeans clustering* — the silhouette profile of Figure 7, with the argument for reading the profile rather than the mean. [free] https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html
- scikit-learn example, *Demonstration of k-means assumptions* — four ways k-means fails: wrong k , anisotropic blobs, unequal variance and unevenly sized blobs. Rows two and three of Section 4.2. [free] https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_assumptions.html
- scikit-learn example, *Color Quantization using K-Means* — the computation behind Figure 12, on the same bundled photograph. [free] https://scikit-learn.org/stable/auto_examples/cluster/plot_color_quantization.html

Articles

- V. Estivill-Castro, “Why so many clustering algorithms: a position paper”, *ACM SIGKDD Explorations Newsletter* **4**(1), 65–75, 2002. The source of the sentence Section 3 is built around: *clustering is in the eye of the beholder*, and the argument that the multiplicity of algorithms follows from the impossibility of defining “cluster” precisely. <https://doi.org/10.1145/568574.568575>
- P. J. Rousseeuw, “Silhouettes: a graphical aid to the interpretation and validation of cluster analysis”, *Journal of Computational and Applied Mathematics* **20**, 53–65, 1987. The paper that defines equation (7) and the silhouette plot of Figure 7. [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7)
- T. Caliński and J. Harabasz, “A dendrite method for cluster analysis”, *Communications in Statistics* **3**(1), 1–27, 1974. The origin of equation (6). <https://doi.org/10.1080/03610927408827101>

- D. L. Davies and D. W. Bouldin, “A cluster separation measure”, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **PAMI-1**(2), 224–227, 1979. Equation (8). <https://doi.org/10.1109/TPAMI.1979.4766909>
- L. Hubert and P. Arabie, “Comparing partitions”, *Journal of Classification* **2**(1), 193–218, 1985. The paper that corrects the Rand index for chance; §2 is the derivation behind equation (9) and behind the 0.5567 against -0.0010 result of Section 6.6. <https://doi.org/10.1007/BF01908075>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Kriegel, Sander and Xu, “A density-based algorithm for discovering clusters (DBSCAN),” KDD 1996.